

- 数据结构期末复习提纲
  - 一、复习总方向
  - 二、图部分：本次复习重中之重
    - 1. 要掌握的核心知识
    - 2. 图的存储结构
    - 3. 图的遍历
      - DFS
      - BFS
    - 4. 路径与最短路
    - 5. 图上的典型题型
    - 6. 图部分考试时的答题模板
    - 7. 图部分必须自己能手写的代码骨架
  - 三、树部分：第二重点
    - 1. 树部分必须掌握的知识
    - 2. 二叉树遍历
    - 3. 树的重构
    - 4. 树上统计问题
    - 5. 树上的典型算法
    - 6. 树部分考试时的思考顺序
  - 四、栈与队列：本次考试的关键辅助结构
    - 1. 栈要会的内容
    - 2. 队列要会的内容
    - 3. 必须形成的“条件反射”
  - 五、递归与迭代：必须能互相转换
    - 1. 递归必须掌握的三件事
    - 2. 哪些题最适合递归
    - 3. 哪些题容易改成迭代
    - 4. 复习建议
  - 六、链表：基础分不能丢
    - 1. 应会内容
    - 2. 这部分常见失分点
  - 七、结合老师提示，需要特别防备的“题目新定义”
    - 1. 遇到新定义时的处理方法
    - 2. 常见情况
  - 八、建议重点复盘的目录文件
    - 图
    - 树

- 栈和队列
- 九、最后冲刺时最应该会写的 12 类程序
- 十、考场答题建议
- 十一、一句话总结

# 数据结构期末复习提纲

---

本文根据老师图片中的期末提示整理，并参考当前目录下的数据结构作业与教材代码，总结出最值得复习的考点、常见题型和答题时的实现套路。内容刻意避开 **leetcode** 方向，尽量贴合你这学期作业里真正出现过的内容。

---

## 一、复习总方向

---

老师提示可以概括为 5 个关键词：

1. 图
2. 树的重构与树上算法
3. 递归 / 迭代构造算法
4. 栈和队列的正确使用
5. 理解题目新定义的非标准概念

结合目录中的文件，建议复习时按下面优先级进行：

1. 图
  2. 树
  3. 栈与队列
  4. 链表基础
  5. 递归、回溯、DFS/BFS 的统一写法
- 

## 二、图部分：本次复习重中之重

---

图片里第一条明确说明：考试会考“用图结构解决实际问题”。这部分必须重点准备。

### 1. 要掌握的核心知识

1. 图的构建
2. 图的存储结构
3. 图的遍历
4. 路径长度计算
5. 用图解决实际问题

## 2. 图的存储结构

必须会区分两种表示：

1. 邻接矩阵
2. 邻接表

要会回答：

1. 各自适合什么图
2. 建图时如何从输入转换
3. 遍历某个顶点的邻接点时怎么写

结合目录中的代码，重点看：

1. [traverse.cpp](#)
2. [init.cpp](#)

## 3. 图的遍历

### DFS

要掌握：

1. 递归写法
2. `visited[]` 的作用
3. 非连通图时如何遍历所有连通分量
4. DFS 如何用于找路径

对应目录文件：

1. [traverse.cpp](#)
2. [liantongfenliang.cpp](#)

## BFS

要掌握：

1. 队列实现
2. 按层推进
3. 无权图最短路径思想
4. 迷宫、最少步数类题目

对应目录文件：

1. [traverse.cpp](#)
2. [migong.cpp](#)

## 4. 路径与最短路

图片里明确提到“路径长度计算”，所以要会：

1. 路径是什么
2. 路径长度怎么定义
3. 无权图最短路为什么优先 BFS
4. 带权图最短路的基本思想

目录中可重点复习：

1. [Floyd.cpp](#)
2. [Floyd.cpp](#)
3. [theSmallestPath.cpp](#)
4. [waterToCity.cpp](#)
5. [scenicRoute1.cpp](#)

你至少要能说清：

1. Floyd 的三重循环含义
2. `dist[i][j]` 表示什么
3. 初始化时不可达如何表示
4. 为什么更新式是  $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$

## 5. 图上的典型题型

根据你的作业，图这部分高频题型有：

1. 图遍历输出
2. 连通分量
3. 最短路径
4. 最小生成树
5. 拓扑排序
6. 冗余边/判环

对应文件建议重点看：

1. [zuixiaoshengchengshu.cpp](#)
2. [buildSmallestTree.cpp](#)
3. [Prim.cpp](#)
4. [topsort.cpp](#)
5. [RedundantConnection.c](#)

## 6. 图部分考试时的答题模板

做图题时建议固定分四步：

1. 读题，判断是有向图还是无向图、带权还是无权图
2. 选存储结构，邻接矩阵还是邻接表
3. 选算法，DFS / BFS / Floyd / Prim / 拓扑排序
4. 最后处理输出格式

## 7. 图部分必须自己能手写的代码骨架

1. 邻接表建图
2. DFS
3. BFS
4. Floyd
5. Prim 或 Kruskal 的基本流程

---

## 三、树部分：第二重点

---

图片第二条明确要求：考察树的重构，以及树上的遍历、路径长度等算法。

## 1. 树部分必须掌握的知识

1. 树的基本概念
2. 二叉树的遍历
3. 由遍历序列重构二叉树
4. 层序建树
5. 树上路径问题
6. 宽度、层数、结点数等统计问题

## 2. 二叉树遍历

必须熟练掌握：

1. 前序遍历
2. 中序遍历
3. 后序遍历
4. 层序遍历

不仅要会递归，也要知道：

1. 前中后序都可以用栈改写
2. 层序遍历天然使用队列

目录中对应文件：

1. [tree2.cpp](#)
2. [buildtree.cpp](#)
3. [levelnum.cpp](#)
4. [iscompletetree.cpp](#)

## 3. 树的重构

这是老师点名要考的内容。

重点掌握两类：

1. 前序 + 中序重构二叉树

## 2. 后序 + 中序重构二叉树

核心思想一定要背熟：

1. 前序第一个结点是根
2. 后序最后一个结点是根
3. 在中序里找到根，就能划分左子树和右子树
4. 再递归构造左右子树

重点文件：

1. [buildtree.cpp](#)
2. [buildtree.cpp](#)

考试中常见问法：

1. 给出前序和中序，构造二叉树
2. 给出后序和中序，构造二叉树
3. 构造后输出某种遍历结果

## 4. 树上统计问题

图片中专门提示“建议仔细研究求二叉树宽度的示例代码”，这一块一定要会。

必须会：

1. 求树高
2. 求第  $k$  层结点数
3. 求叶子结点数
4. 求树的宽度
5. 判断完全二叉树

重点文件：

1. [levelnum.cpp](#)
2. [homework.md](#)
3. [iscompletetree.cpp](#)

这里建议重点记住：

1. 宽度通常用层序遍历做
2. 每层开始时记录当前队列长度

3. 某层的队列长度就是该层结点数
4. 所有层中的最大值就是宽度

## 5. 树上的典型算法

结合你的作业，树上的典型算法包括：

1. 最近公共祖先
2. 路径和
3. 最大路径和
4. 距离为 K 的结点
5. 树转链表
6. 合并树
7. 垂序遍历

可参考：

1. [LCA.c](#)
2. [pathsum.cpp](#)
3. [maxway.cpp](#)
4. [distanceKNode.cpp](#)
5. [TreetoList.cpp](#)
6. [MergeTrees.cpp](#)
7. [verticalTraversal.cpp](#)

## 6. 树部分考试时的思考顺序

树题建议按下面顺序做：

1. 先判断输入给的是哪种树表示
2. 先把树建出来
3. 再判断用哪种遍历最合适
4. 如果要求层、宽度、完全性，优先想到队列和层序遍历
5. 如果要求路径、最大值、祖先，优先想到递归 DFS

---

## 四、栈与队列：本次考试的关键辅助结构

---



图片第三、四条专门强调：

1. 用递归或迭代构造算法
2. 正确使用栈或队列作为辅助结构
3. 允许直接使用 `std::stack` 和 `std::queue`

这说明考试很可能不是单纯考“定义”，而是考“什么时候该想到栈，什么时候该想到队列”。

## 1. 栈要会的内容

1. 栈的基本操作
2. 出栈序列合法性判断
3. 单调栈
4. 用栈模拟递归
5. 表达式求值
6. DFS 中显式栈写法

对应文件：

1. [max\\_pop.c](#)
2. [cal.c](#)
3. [simu\\_product.c](#)
4. [biggest\\_rect.cpp](#)
5. [train.c](#)

## 2. 队列要会的内容

1. 队列的基本操作
2. 循环队列判空判满
3. 层序遍历中的队列
4. BFS 中的队列
5. 队列与栈互相配合

对应文件：

1. [homework.md](#)
2. [iscompletetree.cpp](#)
3. [traverse.cpp](#)

## 3. 必须形成的“条件反射”

看到下面这些词，要立刻想到对应结构：

1. 后进先出、括号、递归改写、回退过程：想到栈
  2. 按层、最少步数、一圈一圈扩展：想到队列
  3. 树的层序遍历：想到队列
  4. 图的 BFS：想到队列
  5. 图或树的 DFS 非递归版：想到栈
- 

## 五、递归与迭代：必须能互相转换

---

老师第三条说得很明确：要会“用递归方式或迭代方式构造算法”。

### 1. 递归必须掌握的三件事

1. 递归参数表示什么
2. 递归终止条件是什么
3. 递归函数返回值表示什么

### 2. 哪些题最适合递归

1. 树的遍历
2. 树的重构
3. 路径搜索
4. 回溯枚举
5. 图的 DFS

### 3. 哪些题容易改成迭代

1. DFS 改显式栈
2. 层序遍历本身就是迭代
3. 递归乘积、递归调用过程可用栈模拟

重点文件：

1. [simu\\_product.c](#)
2. [tree2.cpp](#)
3. [phonenum\\_dfs.cpp](#)

## 4. 复习建议

对每一种遍历，最好都问自己两遍：

1. 递归怎么写
2. 不用递归怎么写

---

## 六、链表：基础分不能丢

虽然老师图片重点不在链表，但你的作业里链表、集合差集、循环链表等内容很多，属于容易拿分的基础题。

### 1. 应会内容

1. 带头结点单链表
2. 无头单链表
3. 双向链表
4. 循环链表
5. 插入、删除、查找
6. 集合差集
7. 倒数第  $k$  个结点

参考文件：

1. [homework.md](#)
2. [S5.cpp](#)
3. [S6.cpp](#)
4. [S7.c](#)

### 2. 这部分常见失分点

1. 忘记处理空表
  2. 头结点和首元结点混淆
  3. 删除结点时断链顺序错误
  4. 忘记释放空间
  5. 循环链表终止条件写错
- 

## 七、结合老师提示，需要特别防备的“题目新定义”

---

图片第五条很重要：考试可能会定义一些非标准概念，例如“二叉树的宽度”等。

这意味着你做题不能死背名词，要：

1. 先认真读定义
2. 看例子
3. 搞清楚输入输出
4. 再开始编码

### 1. 遇到新定义时的处理方法

1. 先把定义翻成你熟悉的操作对象
2. 判断它本质上是树问题、图问题，还是栈队列辅助问题
3. 画一个小样例手推
4. 再决定算法

### 2. 常见情况

1. 实际上是层序统计，但换了名字
  2. 实际上是路径问题，但换了描述
  3. 实际上是 DFS / BFS，只是背景换成迷宫、校园、交通网络
- 

## 八、建议重点复盘的目录文件

---

如果考前时间有限，建议优先看这些文件：

## 图

1. [traverse.cpp](#)
2. [migong.cpp](#)
3. [Floyd.cpp](#)
4. [buildSmallestTree.cpp](#)
5. [topsort.cpp](#)

## 树

1. [buildtree.cpp](#)
2. [levelnnum.cpp](#)
3. [iscompletetree.cpp](#)
4. [tree2.cpp](#)
5. [distanceKNode.cpp](#)

## 栈和队列

1. [cal.c](#)
2. [max\\_pop.c](#)
3. [simu\\_product.c](#)
4. [biggest\\_rect.cpp](#)
5. [train.c](#)

---

## 九、最后冲刺时最应该会写的 12 类程序

---

1. 邻接表建图
2. 图的 DFS
3. 图的 BFS
4. Floyd 最短路
5. 最小生成树
6. 拓扑排序

7. 前序 + 中序重构二叉树
  8. 后序 + 中序重构二叉树
  9. 二叉树层序遍历
  10. 求二叉树宽度 / 第 k 层结点数
  11. 用栈模拟递归或表达式求值
  12. 用队列处理层次问题或最少步数问题
- 

## 十、考场答题建议

---

1. 先判断题目本质属于图、树、栈、队列中的哪一类
  2. 先写数据结构定义，再写主过程
  3. 图题先建图，树题先建树
  4. 一旦涉及“按层”“最少步数”，先想 BFS 和队列
  5. 一旦涉及“回退”“括号”“递归改写”，先想栈
  6. 写完后重点检查边界条件：空图、空树、单结点、不可达、重复访问
- 

## 十一、一句话总结

---

这次期末最值得花时间的主线是：

图的建模与遍历 + 树的重构与层次问题 + 栈/队列作为辅助结构的熟练使用

如果这三条线你都能把代表代码手写出来，基本就抓住了老师提示里的核心。